CS 344 **Problem Set 3**
Fall 2021 **Due:** Nov 22, 2021, 11:59 p.m.

Name: (Xiaowei Zhang) **NetID**: (xz561)

Honor Code: Students may discuss and work on homework problems in groups, which is encouraged. However, each student must write down their solutions independently to show they understand the solution well enough to reconstruct it by themselves. Students should clearly mention the names of the other students who offered discussions. We check all submissions for plagiarism. We take the honor code seriously and expect students to do the same.

Instruction for Submission: This homework has a total of 100 points (+10 bonus points), it will be rescaled to 10 points (+1 bonus point) as the eventual score. We encourage you to use LaTex to write your answer, because it's particularly suitable to type equations and it's frequently used for writing academic papers. We have provided the homework3.tex file for you, you can write your answer to each question in this .tex file directly after the corresponding question, and then compile the .tex file into a PDF file for submission. As a quick introduction to LaTex, please see this *Learn LaTeX in 30 minutes* [1]. Compiling a .tex file into a PDF file needs installing the LaTex software on your laptop. It's free and open source. But if you don't want to install the software, you can just use this website `https://www.overleaf.com/`, which is also free of charge. You can just create an empty project in this website and upload the homework1.zip, and then the website will compile the PDF for you. You can also directly edit your answers on the website and instantly compile your file.

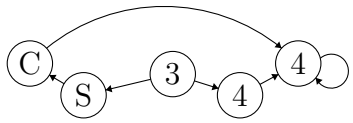
You can also use Microsoft Word or other software if you don't want to use LaTex. If so, please clearly number your answers so that we know which answer corresponds to which question. You also need to save your answers as a PDF file for submission.

Finally, please submit your PDF file only. You should name your PDF file as "Firstname-Lastname-NetID.pdf".

Late Policy: The homework is due on 11/22 (Monday) at 11:59pm. We will release the solutions of the homework on Canvas on 11/26 (Friday) 11:59pm. If your homework is submitted to Canvas before 11/22 11:59pm, there will no late penalty. If you submit to Canvas after 11/22 11:59pm and before 11/26 11:59pm (i.e., before we release the solution), your score will be penalized by 0.9^k, where k is the number of days of late submission. For example, if you submitted on 11/25, and your original score is 80, then your final score will be $80 * 0.9^3 = 58.32$ for $25 - 22 = 3$ days of late submission. If you submit to Canvas after 11/26 11:59pm (i.e., after we release the solution), then you will earn no score for the homework.

Drawing graphs: You might try `http://madebyevan.com/fsm/` which allows you to draw graphs with your mouse and convert it into LaTeX code:

[1] `https://www.overleaf.com/learn/latex/Learn_LaTeX_in_30_minutes`



You can also draw by hand and insert a picture.

Make best use of picture in Latex: If you think some part of your answer is too difficult to type using Latex, you may write that part on paper and scan it as a picture, and then insert that part into Latex as a picture, and finally Latex will compile the picture into the final PDF output. This will make things easier. For instructions of how to insert pictures in Latex, you may refer to the Latex file of our homework 1, which includes several examples of inserting pictures in Latex.

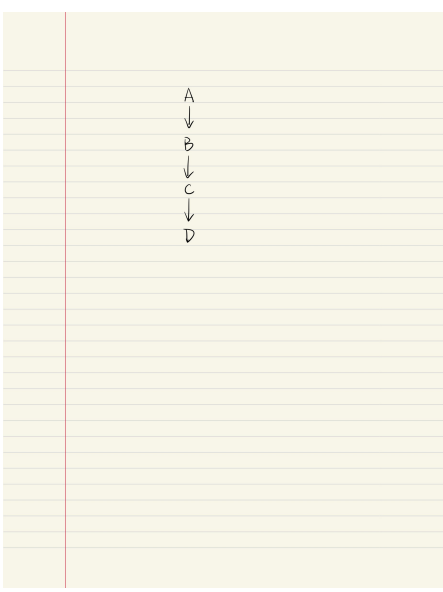
Discussion Group (People with whom you discussed ideas used in your answers if any):

I acknowledge and accept the Honor Code. Please type your initials below:
Signed: (XZ)

---

1. **Warmup with DFS/BFS.** (20 points)

(1) Give one example of a directed graph on four vertices, A, B, C, and D, such that both depth-first search and breadth-first search discover the vertices in the same order when started at A.



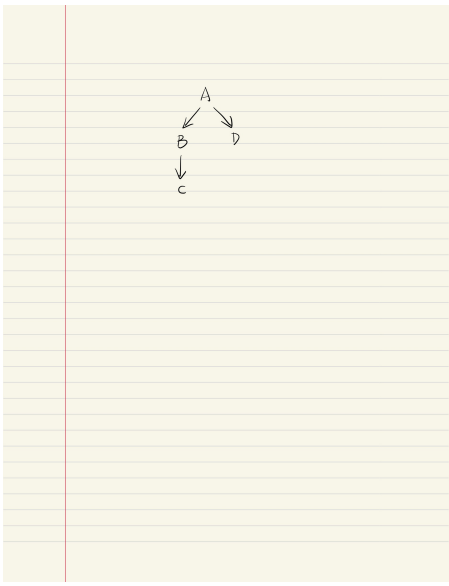
Answer:

DFS:A,B,C,D

BFS:A,B,C,D

(2) Give one example of a directed graph where DFS and BFS discover the vertices in a different order when started at A.

Answer:



DFS:A,B,C,D

BFS:A,B,D,C

"Discover" means the time that the algorithm first reaches the vertex, referred to as `start_time` during lecture. Assume that both DFS and BFS iterate over outgoing neighbors in alphabetical order.

[We are expecting a drawing of your graphs and an ordered list of vertices discovered by DFS and BFS.]

2. **Warmup with Dijkstra.** (25 points)

   Let $G = (V, E)$ be a weighted directed graph. For the rest of this problem, assume that $s, t \in V$ and that **there exists a directed path from s to t.**

   For the rest of this problem, refer to the implementation of Dijkstra's algorithm given by the pseudocode below.

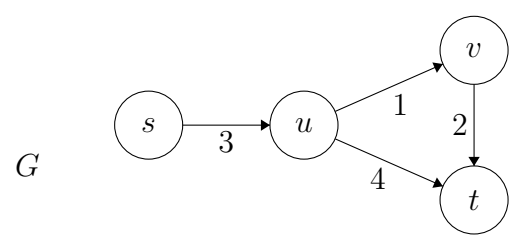
```
dijkstra_st_path(G, s, t):
  for all v in V, set d[v] = Infinity
  for all v in V, set p[v] = None

  // we will use p to reconstruct the shortest s-t path at the end
  d[s] = 0
  F = V
  D = []
  while F isn't empty:
    x = vertex v in F such that d[v] is minimized
    for y in x.outgoing_neighbors:
      d[y] = min( d[y], d[x] + weight(x,y) )
      if d[y] was changed in the previous line, set p[y] = x
    F.remove(x)
    D.add(x)

  // use p to reconstruct the shortest s-t path
  path = [t]
  current = t
  while current != s:
    current = p[current]
    add current to the front of the path
  return path, d[t]
```

The variable `p` maintains the "**p**arents" of the vertices in the shortest s-t path, so it can be reconstructed at the end.

Step through `dijkstra_st_path`(G, s, t) on the graph G shown below. Complete the table below to show what the arrays `d` and `p` are at each step of the algorithm, and indicate what path is returned and what its cost is.



[We are expecting the table below filled out, as well as the final shortest path and its cost. No further justification is required.]

	d[s]	d[u]	d[v]	d[t]	p[s]	p[u]	p[v]	p[t]
When entering the first while loop for the first time, the state is:	0	∞	∞	∞	None	None	None	None
Immediately after the first element of D is added, the state is:	0	3	∞	∞	None	s	None	None
Immediately after the second element of D is added, the state is:	0	3	4	7	None	s	u	u
Immediately after the third element of D is added, the state is:	0	3	4	6	None	s	u	v
Immediately after the fourth element of D is added, the state is:	0	3	4	6	None	s	u	v

Answer: path is [s, u, v, t] and cost is 6

3. **Fun with Reductions.** (15 points) Suppose the economies of the world use a set of currencies $C_1, \ldots, C_n$; think of these as dollars, pounds, Bitcoin, etc. Your bank allows you to trade each currency C_i for any other currency C_j, and finds some way to charge you for this service. Suppose that for each ordered pair of currencies (C_i, C_j), the bank charges a flat fee of $f_{ij} > 0$ dollars to exchange C_i for C_j (regardless of the quantity of currency being exchanged).

   Describe an algorithm which, given a starting currency C_s, a target currency C_t, and a list of fees f_{ij} for all $i, j \in \{1, \ldots, n\}$, computes the cheapest way (that is, incurring the least in fees) to exchange all of our currency in C_s into currency C_t. Also, justify the its runtime.

   [We are expecting a description or pseudocode (either is OK) of your algorithm, as well as a brief justification of its runtime. You can use any algorithm we have learned in class.]

   Answer: We can treat this problem as finding the shortest path from C_s to C_t where all the weights of each path between two currencies > 0. Thus we use dijkstra algorithm here.

   C is the list of currencies, s is the starting currency and t is the ending currency

```
dijkstra_st_path(C, s, t, f): //f is the list of flat fees
  for all c in C, set d[c] = Infinity
  for all c in C, set p[c] = None

  // we will use p to reconstruct the shortest s-t path at the end
  d[s] = 0
  F = C
  D = []
  while F isn't empty:
```

```
    x = currency c in F such that d[c] is minimized
    //we find x's outgoing neighbors by go through all pairs of currencies
    for y in x.outgoing_neighbors:
      //f(x,y) is the flat fee charged by exchanging x for y
      d[y] = min( d[y], d[x] + f(x,y))
      if d[y] was changed in the previous line, set p[y] = x
    F.remove(x)
    D.add(x)

  // use p to reconstruct the shortest s-t path
  path = [t]
  current = t
  while current != s:
    current = p[current]
    add current to the front of the path
  return path, d[t]
```

Run time: suppose we implement the list d with a binary min-heap, adding all currencies to d will take O($|C|$) where $|C|$ is the number of currencies in the list. In the first while loop, F contains all currencies and each currency will be removed once, thus there are $|C|$ iterations. In each iteration, removing the currency with minimized d value from the binary heap will take O(log$|C|$). Hence, the removal of all vertices from d is $O(|C|log|C|)$. For all y as x's outgoing neighbors, the total number of y is bounded by $|E|$ where $|E|$ is the number of ordered pair of currencies. And to update d[y] in the binary heap takes $O(log|C|)$ assuming that we keep a hash map of currencies and their respective indices in the binary heap array and assume the put operation takes constant time. We also assume that f is implemented with an adjacency list so access f(x,y) will take constant time, which is the same as accessing d[x] in binary min-heap. The total run time of first while loop should be $O((|C|+|E|)log|C|)$. The second while loop will iterate at most $|C|$ times, thus the total run time is $2O(|C|) + O((|C| + |E|)log|C|)$, which is the same as $O((|C| + |E|)log|C|)$

4. **Social engineering.** (20 points)

   Suppose we have a community of n people. We can create a directed graph from this community as follows: the vertices are people, and there is a directed edge from person A to person B if A would forward a rumor to B. Assume that if there is an edge from A to B, then A will always forward any rumor they hear to B. Notice that this relationship isn't symmetric: A might gossip to B but not vice versa. Suppose there are m directed edges total, so $G = (V, E)$ is a graph with n vertices and m edges.

   Define a person P to be *influential* if for all other people A in the community, there is a directed path from P to A in G. Thus, if you tell a rumor to an influential person P, eventually the rumor will reach everybody. You have a rumor that you'd like to spread, but you don't have time to tell more than one person, so you'd like to find an influential person to tell the rumor to.

In the following questions, assume that G is the directed graph representing the community, and that you have access to G as an array of adjacency lists: for each vertex v, in $O(1)$ time you can get a pointer to the head of the linked lists v.`outgoing_neighbors` and v.`incoming_neighbors`. Notice that G is not necessarily acyclic. In your answers, you may appeal to any statements we have seen in class, in the notes, or in CLRS.

(a) (10 points) Show that all influential people in G are in the same strongly connected component, and that everyone in this strongly connected component is influential.

[Hint: You need to refer the definition of strongly connected component, and you can prove using either induction or contradiction.]

Answer:The definition of strongly connected components says that A directed graph G=(V,E) is strongly connected if, for all pairs of vertices u and v in G, there's a path from u to v and a path from v to u.

We prove "all influential people in G are in the same strongly connected component" by contradiction. Suppose there is an influential person p that belongs to another component G1 that is different from the strongly connected component G2 which contains all the rest influential people. Because G1 and G2 are different, either there isn't a path from G1 to G2, or there isn't a path from G2 to G1, or both. Since p is an influential person, there is a directed path from p to any other people in the community. Therefore, there are directed paths from p to all the people in G2. Also, since all the people in G2 are influential, there are directed paths from every person in G2 to p, i.e. G1. Hence, we come to a contradiction. Therefore, all influential people in G are in the same strongly connected component.

We prove "everyone in this strongly connected component is influential" by contradiction. Suppose there is a person p in the strongly connected component as with all the other influential people. Suppose this person is not influential. Then by the definition of influential, there should be not any directed path from p to other influential people in the strongly connected component. Because if there is and suppose that influential person to whom p is connected is f, p will be indirectly connected to all the other people in the community through f. Since there is no path from p to any other people in the strongly connected component, p is not in the SCC. Hence we come to our contradiction. Therefore, everyone in this strongly connected component is influential

(b) (10 points) Suppose that an influential person exists. Give an algorithm that, given G, finds an influential person.

[We are expecting a description or pseudocode of your algorithm and a short argument about the runtime. You can borrow any algorithm that we have learned in class.]

Answer: We are using Kosaraju's algoithm here. Since all the influential people are in the same SCC. Finding the SCC will find the influential person.

```
Kosaraju(G):
    for all v in V that visited[v] = false:
        DFS(v){
```

```
            visited[v] = true;
            for all outgoing neighbors X of v that is not visited:
                DFS(X)
            //the top of the stack should be the last vertex visited
            stack.push(v)
        }
    return stack.pop()
```

We know that the SCC that consists only influential people should be the SCC contains the vertex with the largest ending time. Because between two SCCs G1 and G2, there is either a directed path from G1 to G2 or from G2 to G1 or none. For the SCC with all influential people, there should be always a directed path from the SCC to other SCCs. Therefore, only the SCC with no incoming edges, that is, contains vertex with the largest ending time will fulfill the requirement. Our goal is to find one influential person, the vertex on top of the stack will fulfill this requirement. Time complexity is O(m+n) because DFS will run at most n times for each vertex in V and the total number of testing whether X is visited or not is m times. The total run time will be like $|v_1| + |v_1$'s outgoing neighbors$| + |v_2| + |v_2$'s outgoing neighbors$| + ... + |v_n| + |v_n$'s outgoing neighbors$| = O(|V| + |E|) = O(n+m)$

5. **Job Sequencing Problem.** (10 bonus points)

In this problem we have n jobs $j_1, j_2, ..., j_n$, each has an associated deadline $d_1, d_2, ..., d_n$ and profit $p_1, p_2, ..., p_n$. Profit will only be awarded or earned if the job is completed before the deadline. We assume that each job takes 1 unit of time to complete. The objective is to earn maximum profit when only one job can be scheduled or processed at any given time.

Describe or provide the pseudocode of an algorithm to find the sequence of jobs to do with the maximum total profit.

Answer: A greedy algorithm will be useful here.

1.Sort the jobs based on the profit in a non-increasing order.

2.for every job j in the non-increasing order, find a time slot i such that $i < deadline(j)$ and i is the max possible value. Place the job in the slot and mark the slot as full

3. if no such slot exist, ignore the job.

4. return the time slots, maximum profit as the sum of all profits of all jobs in the time slots

[Hint: You can select the jobs in a greedy way. You can use the following example to help your analysis.]

Job	J1	J2	J3	J4	J5
Deadline	2	1	3	2	1
Profit	60	100	20	40	20

The best job sequence would be $J2 \rightarrow J1 \rightarrow J3$.

6. **Alternative Minimum Spanning Trees (23.4 CLRS).** (20 points) In this problem, we give pseudocode for two different algorithms. Each one takes a connected graph and a weight function as input and returns a set of edges T. For each algorithm, either show that T is a minimum spanning tree (give a brief justification of the algorithm) or show that T is not a minimum spanning tree (give a counter-example).

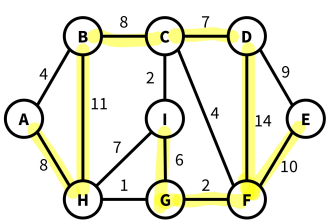
a. (10 points) **MAYBE-MST-A** (G, w)

1. sort the edges into non-increasing order of edge weights w
2. $T = E$
3. for each edge $e \in E$, taken in non-increasing order by weight
4. if $T - \{e\}$ is a connected graph
5. $\quad T = T - \{e\}$
6. return T

Answer: T is indeed a MST. At every iteration of edge in edges, we are always removing the edge e that by its removal the graph will still be connected. That is, the edge has to be a part of a cycle in the graph. Also, from the cut's perspective, since removing e won't induce a disconnected graph, that means e is not the only edge that connects two components between the cut for any cut that crosses e. Thus, removing e is safe. We are removing all the edges that are safe, so the remaining graph should contain no cycle at all but still connects all vertices. And since we are removing the edges in the non-increasing order, all remaining edges in every particular cycle should have weight less or equal to the weight of that removed edge. So the result should yield a Minimum Spanning Tree

b. (10 points) **MAYBE-MST-B** (G, w)

1. $T = null$
2. for each edge e, taken in arbitrary order
3. $\quad$ if $T \cup \{e\}$ has no cycles
4. $\quad T = T \cup \{e\}$
5. return T

Answer: T is not a MST.

A counter example is given below, where yellow edges construct a spanning tree. suppose e is in order DF,EF,GF,GI,CD,BC,BH,AH and the rest. The resulting spanning tree is not a MST.